Introduction to parallel computing and OpenMP in C++ Part 1: Parallelizing a `for` loop

Note to all students: Please keep track of how much time you spend on this assignment. Your time commitment will be one of the deliverables.
Note to on-line students: This activity took the live class 2 class periods to complete. They did have the advantage of asking me questions while they were working. Please feel free to email me with questions.

This activity is (ideally) a pair programming exercise. This means that one person is at the keyboard (the driver) and the other is the navigator, observing, reviewing, and offering advice. Be sure to switch rolls at least once (maybe around question 12). This activity will be an introduction to parallel programming in general and using the OpenMP API in C++ by taking a look at a way to parallelize the iterations in a loop. This activity will also examine one method of measuring the performance of a piece of code by timing how long it takes to run. **You may use AI or a web search to find the answers for all the questions where permitted.** AI is an excellent tool for getting started learning about a new topic. Each part will require you to write out an answer, so make a copy of this document to write your team's answers. (If you are not in the live class and/or don't have a partner for this activity, just do it on your own.)

NOTE: The use of OpenMP probably will not work with a cloud-based environment like CodeHS and using OpenMP will require a few additional steps on a MacOS. It will work great with a Raspberry Pi running linux.

Part 1: Learn about the OpenMP API

OpenMP is an API that supports shared-memory parallel processing for C/C++ and Fortran. .

1. Define API [AI]

2. Define parallel processing [AI]

3. Define shared-memory [AI]

OpenMP allows programmers to parallelize code by adding pragmas to existing code.

4. What's a `pragma`? [AI]

5. What is the goal of parallelization of code? [AI]

6. What's a "core"? [AI]

7. How many cores does the computer you are using have? Some ideas might be to find out what kind of processor your computer has (usually buried in a 'settings' tab). Or if

you are on a linux machine you can explore the commands `nproc`, `lscpu`, and `htop` or `top`. [AI]

The programming we have done so far in COSC 1010 and COSC 1030 has been sequential.

8. What does “sequential processing" mean? [AI]

Part 2: Write a short program that will hopefully take a while if run sequentially. Then explore a few aspects of OpenMP to parallelize a section of the code.

In parallel computing, the goal is to split up the task into subtasks that can be performed by different processors. OpenMP is an API that allows a programmer to use compiler directives to specify parts of the code that should run in parallel.

9. Write a program that allocates memory for 100 million integers and a loop that assigns each index in the loop to contain the integer corresponding to its index. So at index 0 the value in the array is 0, at index 1, the array is 1 and so forth. Note that you might have to explore the `long` and `long long` data types since the numbers we are dealing with here can be quite long. Feel free to use AI tools for this task to find out what the largest integer that can be stored on a given system. Remember that these values are not standard for C++, so you will need to figure out what those maximum values are on the particular system you are using. Also, remember that your system may not be able to allocate this much stack space, which will require using heap memory through dynamic memory allocation (using pointers). I encourage you to experiment with trying the program with stack memory to see what happens.

10. Once you have written the program in 10, consider the loop that fills the array. Is this an example of sequential processing or parallel processing?

11. Could this program be parallelized? Could tasks in this program be done by different cores at the same time? (Think back to the penny searching experience.)

12. The compiler directive to parallelize a loop is:

    ```
    #pragma omp parallel for
    ```

    The directive is placed directly above the `for` line of the loop that fills the array. Note that this pragma works for `for` loops and not while loops, so if you wrote the loop as a `while` loop, re-write it as a for loop. You will also need to add an include:

    ```
    #include <omp.h>
    ```

    To compile the program, you will need to add a new compiler option `-fopenmp`. If the source code is located in the file main.cpp, the command to compile will be:

```
g++ -fopenmp main.cpp -o main
```

Note that using OpenMP on a Mac requires some setup because the default clang compiler on MacOS does not support OpenMP by default. Follow these directions to set up a Mac.

13. There is another way to find out how many processors your computer has. You will need to include the `<thread>` library.

```
#include <thread>
```

The value `thread::hardware_concurrency()` stores the number of processors. Add a line of code to print this out and see if it agrees with the answer you found above.

14. The `#pragma omp parallel for` basically splits the iterations of the loop so that they will be done in parallel by different threads. A thread is like a mini-program. To get to see how this works first reduce the size of the array (and the number of iterations of the loop) to 50. We are going to add a print statement to the loop and we don't need to watch millions of iterations. Inside the loop add the line of code:

```
int id = omp_get_thread_num();
```

The function `omp_get_thread_num()` returns the thread number that is executing an iteration of the loop.

Print out the `id` inside the loop. How many different numbers are printed out? Does this make sense given the number of processors?

15. Add some additional text to your cout statement such as

```
cout << "id: " << id;
```

Does the output seem all jumbled up? Why do you think that could be happening?

To fix this problem, we can change to using a different function called printf(), that is actually part of the C programming language. The equivalent of the above cout using printf is:

```
printf("id: %d", id);
```

The %d is a format specifier for an integer. Try this out and confirm that it fixes the problem.

16. Now compile and run the code with and without the `#pragma omp parallel for`. How many thread ids do you see in both cases?

17. The #pragma omp parallel for is separating the loop into different chunks so that each chunk can be done by a different processor simultaneously (in parallel). Which loop indexes are being handled by each thread? (Hint: print it out and see)

18. Sometimes we want the threads to each have their own copy of a variable and sometimes we want all the threads to share a copy. Are all the threads sharing the indexing variable for the loop, or do they each have their own copy in this case? To find out, print out the memory location where the indexing variable is stored. The format specifier for a memory address/pointer is `%p`, use this to see what the address is for the indexing variable inside the loop. You should find that each thread has it's own memory address for the indexing variable if you have written the loop using something like `for (long long ind = 0; ind < SIZE; ind++)`. What do you think would happen if each thread/process had to share the same memory location for the indexing variable? (HINT: It is the same problem as what happened in #15 when all the threads were competing for a shared resource, printing to the same terminal window. When threads/processes are competing for a resource and it causes unpredictable outcomes depending on which thread or process gets there first, this is called a **race condition** and is generally considered undesirable!)

19. Given the number of processors on your machine, how much faster do you think this task should be if we use all of the processors?

Part 3: Explore the `chrono` library and measure if the parallelization is producing any speedup in the code's runtime. .

20. [AI] To actually measure how fast the code runs we need to take a look at another C++ library `chrono`. Use AI to find out about how to use this library to measure the execution time of a piece of code. The <chrono> library has several different types of clocks, so make sure you have a good understanding of what the different clocks are. Justify your choice of clocks. (Maybe try an AI prompt like "Help me learn to use <chrono> to benchmark my code".)

21. Figure out how to time the block of code that parallelizes the loop. Run the program several times using the #pragma omp parallel for and without using it. Are the times consistent for the same conditions? If not, why? The parallel for loop should be faster than the non-parallel version. Does it come close to the speed up that you predicted above in question 19? If not, why not? (You can use AI to help answer this question.)

22. How much time did this assignment take? Any other comments on how to make this assignment better for future students?

TURN IN: Your answers to the questions above and the code that you wrote and also how long this assignment took and any feedback that will make it better for future students.